Face Mask Detection using Deep Learning (CNN)

A Comparative Study of VGG16 and MobileNetV2 Architectures

A Report submitted

in partial fulfillment of the Spring Project

for the Degree of

Master of Science

By

Sahil Verma

Roll No: 24MA40023

Under The Guidance Of

Prof. Pawan Kumar

Department of Mathematics

Indian Institute of Technology Kharagpur

West Bengal – 721302, India

Department of Mathematics
Indian Institute of Technology
Kharagpur
India – 721302

Certificate of Project

This is to certify that **Sahil Verma**, Roll Number: **24MA40023**, a MSc student in the **Department of Mathematics**, has successfully completed his project titled "**Face Mask Detection using Deep Learning (CNN)**" as part of his **course work** at the **Indian Institute of Technology Kharagpur**.

The project, carried out under my supervision, involved a detailed study of convolutional neural network architectures, specifically focusing on the comparative analysis of **VGG16** and **MobileNetV2**, both implemented and trained from scratch. He developed an end-to-end pipeline for real-time classification, implementing advanced regularization techniques and out-of-core data handling to ensure model robustness. His work demonstrates a strong understanding of applied machine learning, numerical optimization, and computer vision. The presentation was systematic, mathematically precise and reflects consistent effort throughout the project.

Prof. Pawan Kumar (Supervisor)

Department of Mathematics

Indian Institute of Technology Kharagpur

Kharagpur – 721302

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prof. Pawan Kumar** for giving me the opportunity to undertake my M.Sc. project under his guidance on the topic "**Face Mask Detection using Deep Learning (CNN)**". His constant support, valuable suggestions and deep mathematical insight were instrumental throughout the course of this work.

I am also thankful to **Indian Institute of Technology Kharagpur** for providing a stimulating academic environment and the necessary resources for mathematical learning and research.

This project presents a study of convolutional neural network (CNN) architectures and their application in computer vision. It focuses on the design and training of **VGG16** and **MobileNetV2** from scratch for binary image classification. The work involves building the complete network architectures from the ground up, implementing custom dense layers, and evaluating the trade-offs between computational efficiency and classification accuracy.

Finally, I again extend my heartfelt thanks to **Prof. Pawan Kumar** for his valuable mentorship throughout the project.

Sahil Verma
Indian Institute of Technology Kharagpur
Date : 05 - 05 - 2026

ABSTRACT

This project explores the implementation of automated safety monitoring systems using **Deep Learning** frameworks. The primary objective is the development of a robust **Face Mask Detection** system capable of real-time binary classification. To achieve this, two distinct Convolutional Neural Network (CNN) architectures — **VGG16** and **MobileNetV2** — were designed and trained **from scratch** without the use of any pre-trained weights.

Both architectures were built and trained entirely on the face mask dataset, allowing the models to learn task-specific visual features directly from the data. Custom classification heads, including a **Dense layer** with 128 neurons (ReLU activation) and a **Dropout layer** (0.5), were incorporated to mitigate overfitting. The final classification is performed via a **Sigmoid** activation function, outputting a probability $\hat{y} \in [0, 1]$.

The experimental results demonstrate the practical challenges of training deep architectures from random initialization on a limited dataset. **VGG16**, trained for 45 epochs, achieved a stable training accuracy of approximately **99%** and validation accuracy of approximately **97%**, exhibiting consistent convergence. **MobileNetV2**, trained for 50 epochs, exhibited a delayed convergence pattern — with validation accuracy remaining near chance level (50%) for the first 30 epochs before rapidly converging to approximately **97.5%** in later epochs. This behavior is analyzed as a key finding of the study, highlighting the sensitivity of lightweight architectures to initialization and learning dynamics when trained without pre-trained weights.

The system was successfully deployed using an **OpenCV** and **Streamlit** pipeline, enabling real-time inference. This research demonstrates the practical application of numerical optimization and neural network design in public health and safety technology, while also providing insight into the convergence behavior of deep architectures under from-scratch training conditions.

Keywords: Face Mask Detection, Convolutional Neural Networks, Training from Scratch, VGG16, MobileNetV2, Convergence Analysis, Streamlit.

Contents

1 Introduction

1.1 Project Overview

This project focuses on the development of a **Face Mask Detection** system utilizing Deep Learning, specifically Convolutional Neural Networks (CNN). The core objective is to automate the identification of individuals wearing face masks versus those who are not, utilizing real-time camera feeds or static image inputs.

1.2 Motivation and Application

The emergence of global health crises, such as the COVID-19 pandemic, has necessitated the strict enforcement of safety protocols. This system is designed for deployment in high-traffic environments, including: **Public Spaces and Transport, Healthcare Facilities, Corporate and Travel Hubs**

1.3 Problem Formulation

The model functions as a binary classifier that processes an input image—sourced from a curated dataset or a live webcam—and maps it into one of two distinct categories:

1. **With Mask:** The presence of a protective face covering is detected.
2. **Without Mask:** No face covering is identified.

From a mathematical perspective, this task involves defining a mapping function $f : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X} \subset \mathbb{R}^d$ represents the high-dimensional input image space, and $\mathcal{Y} \in \{0, 1\}$ represents the discrete label space. The goal of the CNN is to optimize this mapping to minimize classification error across the training manifold.

2 Dataset and Mathematical Preprocessing

2.1 Dataset Classification

The dataset is structured into two primary categories representing the target classes for the binary classification task:

1. **with_mask:** Images of individuals wearing face coverings.
2. **without_mask:** Images of individuals without face coverings.

The total dataset comprises approximately 8,000 samples, providing a balanced distribution for feature extraction.

2.2 Data Handling: Out-of-Core Learning

Definition

Out-of-core learning refers to a set of algorithms and techniques designed to process datasets that are too large to fit into a computer's main memory (RAM). Instead of loading the entire dataset at once, the system reads small segments of data from the storage disk, processes them, and then discards them to make room for the next segment.

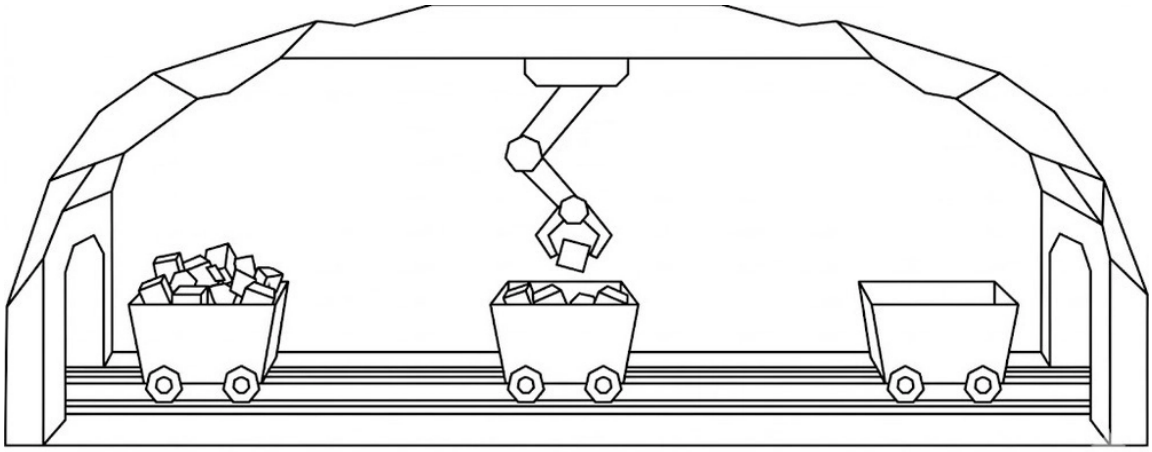

Figure 1: Out of Core Learning

Data Acquisition and Drive Integration

To facilitate out-of-core learning in the Google Colab environment, the dataset is hosted on Google Drive. We establish a virtual link between the cloud storage and the runtime environment using the `google.colab` library. This allows the model to treat the cloud directory as a local file system, enabling the `flow_from_directory` method to stream batches directly from the drive without requiring a full local download to the volatile RAM.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
dataset_path = "/content/drive/MyDrive/Deep Learning Projects/data"
```

```
import os
os.listdir("/content/drive/MyDrive/Deep Learning Projects/data")
```

```
['with_mask', 'without_mask']
```

Figure 2: Drive Integration.

```
for catgory in categories:
  path = os.path.join(dataset_path, catgory)
  for file in os.listdir(path):
    img_path=os.path.join(path,file)
    print(img_path)
    break
```

```
/content/drive/MyDrive/Deep Learning Projects/data/with_mask/with_mask_3456.jpg
/content/drive/MyDrive/Deep Learning Projects/data/without_mask/without_mask_3382.jpg
```

Figure 3: Implementation of Path.

Implementation and Benefits

In this project, processing 8,000 high-resolution images ($224 \times 224 \times 3$) poses a significant memory challenge, as a full load would exceed standard hardware capacities. We address this by implementing the Keras `ImageDataGenerator` to facilitate out-of-core learning. This approach acts as an iterator that fetches and processes images from the disk in real-time, one batch at a time. This not only ensures efficient memory management but also allows for dynamic data augmentation, enabling the model to learn from a diverse range of image variations without requiring additional physical storage space.

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
```

```
train_data = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    batch_size=32,
    class_mode='binary',
    subset='training'
)
```

```
Found 6046 images belonging to 2 classes.
```

```
val_data = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)
```

```
Found 1510 images belonging to 2 classes.
```

Figure 4: Keras Data Generators for out-of-core batch processing.

2.3 Data Partitioning

To ensure the model generalizes well to unseen data, the dataset is partitioned using an 80/20 split ratio:

-Training Set (80%): Approximately 6,046 images used to optimize the weights and biases of the CNN.
-Validation Set (20%): Approximately 1,510 images used to monitor the generalization error and tune hyperparameters.

2.4 Preprocessing and Regularization Strategy

2.4.1 Data Preprocessing Steps

Before the images are propagated through the neural network, they undergo a two-stage mathematical transformation to ensure compatibility with the model architecture:

- **Spatial Standardization:** Every raw input image I is resized to a uniform dimension of $224 \times 224 \times 3$ (RGB channels).
- **Pixel Rescaling:** Raw pixel intensities x, which typically range in the integer interval $[0, 255]$, are transformed into a floating-point interval $[0, 1]$ using the scalar multiplication $x' = x \cdot \frac{1}{255}$.

2.4.2 Theoretical Justification for Preprocessing

The primary objective of these transformations is to improve the efficiency of the gradient descent algorithm. In deep learning, unnormalized inputs can lead to a highly elongated loss surface (high condition number), which causes the gradient updates to oscillate and slows down convergence. By scaling the features to a uniform range, we ensure that the weight updates are more balanced across all layers, leading to a more stable and faster optimization process.

2.4.3 Addressing Overfitting via Regularization

Given that deep CNNs like VGG16 have millions of parameters, they are highly susceptible to "memorizing" the training data rather than learning generalizable features. To prevent this, the following regularization technique is employed:

- **Stochastic Data Augmentation:** During the out-of-core loading process, images are subjected to random transformations such as rotations and horizontal flips. Mathematically, this perturbs the training manifold, forcing the model to learn invariant features (e.g., a face is still a face regardless of its angle).

- **Dropout Regularization:** A Dropout layer with rate 0.5 is applied in the fully connected layers to randomly deactivate neurons during training, preventing co-adaptation and promoting more generalized feature learning.

3 Models and Architecture

3.1 From-Scratch Implementation

In this project, two state-of-the-art Convolutional Neural Network (CNN) architectures, namely **MobileNetV2** and **VGG16**, were designed and trained **from scratch** for the task of *Face Mask Detection*. Unlike transfer learning approaches, no pre-trained weights were utilized. Instead, all network weights were initialized randomly and optimized entirely on the face mask dataset, allowing the models to learn task-specific features directly from the training data.
Both models were built with `weights=None`, meaning the full network — including all convolutional and classification layers — was trained end-to-end from random initialization. This approach requires more training data and epochs to converge, and as observed experimentally, can lead to delayed or non-monotonic convergence behavior in certain architectures, particularly MobileNetV2.
The input image size used for both architectures was:

$$224 \times 224 \times 3$$

where:

- 224 represents image height,
- 224 represents image width,
- 3 denotes RGB color channels.

3.1.1 MobileNetV2

MobileNetV2 is a lightweight deep learning architecture developed for mobile and embedded vision applications. The model is computationally efficient because it utilizes:

- Depthwise Separable Convolutions,
- Inverted Residual Blocks,
- Linear Bottleneck Layers.

These architectural improvements significantly reduce the number of parameters and computational operations while maintaining high classification accuracy.
In this project, the MobileNetV2 architecture was built and trained from scratch on the face mask dataset. All convolutional weights were initialized randomly and updated during training via backpropagation. The network learns facial features such as edges, textures, and structural patterns entirely from the training data without relying on any external pre-trained knowledge.

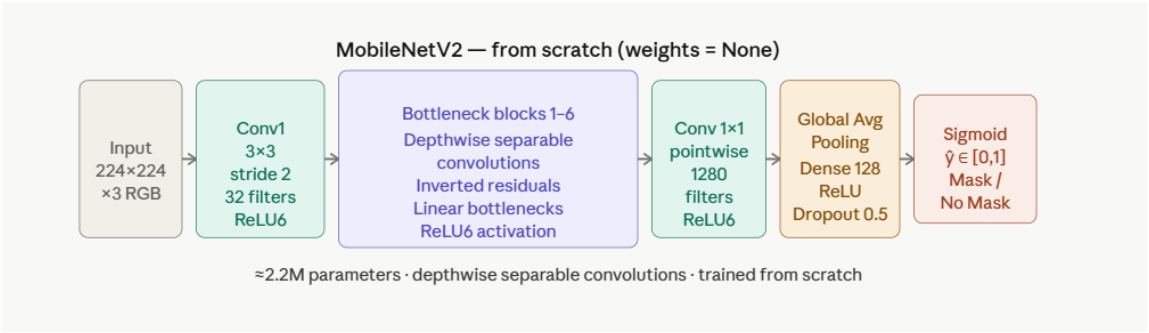

Figure 5: MobileNetV2 architecture trained from scratch — Conv1 layer followed by six depthwise separable bottleneck blocks, Global Average Pooling, Dense(128) + Dropout(0.5) classification head, and a Sigmoid output for binary mask detection.

3.1.2 VGG16

VGG16 is a deep convolutional neural network developed by the Visual Geometry Group (VGG) at the University of Oxford. The architecture is widely recognized for its simplicity and strong feature extraction capability using stacked convolutional layers with small 3×3 kernels.
The VGG16 architecture consists of:

- 13 Convolutional Layers,
- 5 Max Pooling Layers,
- 3 Fully Connected Layers.

In this project, the VGG16 architecture was constructed and trained entirely from scratch. All weights were randomly initialized and learned through the training process on the face mask dataset. The model leverages its deep stack of convolutional layers to progressively extract spatial features — from low-level edges to high-level facial structures — necessary for distinguishing masked from unmasked faces.

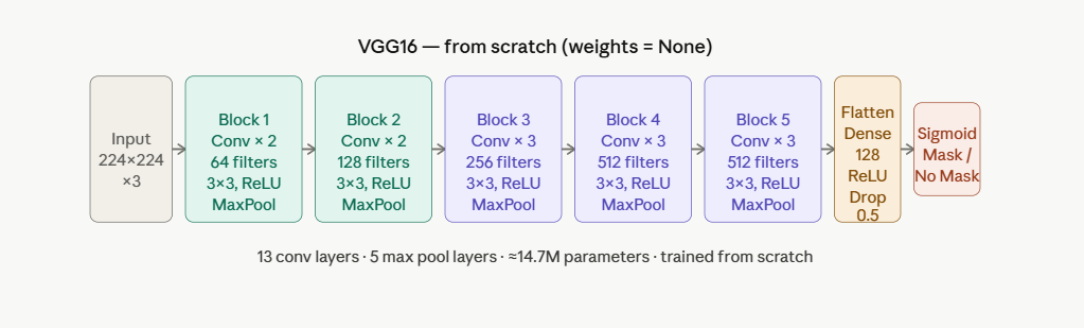

Figure 6: VGG16 architecture trained from scratch — five convolutional blocks (13 conv layers, 64–512 filters), followed by a Dense(128) + Dropout(0.5) classification head and a Sigmoid output for binary mask detection.

3.2 Custom Classification Architecture

To adapt both architectures for binary classification, a custom classification head was appended on top of the convolutional base. Since both models are trained from scratch, all layers — including the convolutional base and the classification head — are fully trainable. The architecture implemented in the project notebook consists of the following layers:

1. **Flatten Layer**

   The Flatten layer converts the multi-dimensional feature maps generated by the convolutional base into a one-dimensional feature vector. This transformation allows the extracted spatial features to be processed by fully connected layers.

2. **Dense Layer (128 Neurons, ReLU Activation)**

   A fully connected dense layer containing 128 neurons was used to learn high-level feature representations associated with face mask detection. The Rectified Linear Unit (ReLU) activation function introduces non-linearity into the model and improves learning efficiency.

   The ReLU function is mathematically expressed as:

   $$f(x) = \max(0, x)$$

3. **Dropout Layer (0.5)**

   A Dropout layer with a dropout rate of 0.5 was added to reduce overfitting. During training, 50% of neurons are randomly deactivated, forcing the network to learn more generalized and robust features.

4. **Output Layer (Sigmoid Activation)**

The final output layer contains a single neuron with a Sigmoid activation function because the problem is a binary classification task.

The Sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The output probability lies within the range:

$$0 \leq P \leq 1$$

where:

- Output close to 1 indicates **Mask**,
- Output close to 0 indicates **No Mask**.

3.3 Implementation of Custom Classification (Code)

Both MobileNetV2 and VGG16 were built from scratch with randomly initialized weights. The following Python snippet illustrates the configuration for MobileNetV2:

```
base_model = MobileNetV2(weights=None, include_top=False,
                         input_shape=(224, 224, 3))
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation="relu")(x)
x = Dropout(0.5)(x)
output = Dense(1, activation="sigmoid")(x)
```

3.4 Model Compilation and Training

Both models were compiled using the **Adam Optimizer** and **Binary Cross-Entropy Loss Function**. Adam optimization combines the advantages of momentum and adaptive learning rate techniques, enabling faster convergence during training.
The Binary Cross-Entropy loss function used in the project is mathematically represented as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where:

- y_i represents the actual class label,
- $\hat{y}_i$ represents the predicted probability,
- N denotes the total number of training samples.

Since both models are trained from scratch, all weights are learned entirely from the face mask dataset. VGG16 was trained for **45 epochs** and MobileNetV2 for **50 epochs**. The additional epochs for MobileNetV2 were necessary to allow sufficient convergence from random initialization, as lightweight architectures with depthwise separable convolutions can exhibit slower initial learning dynamics.

3.5 Performance Evaluation Metrics

The performance of both architectures was evaluated using standard classification metrics: Accuracy, Precision, Recall, F1-Score.
The classification accuracy is computed as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where: TP = True Positive, TN = True Negative, FP = False Positive and FN = False Negative.

3.6 Comparative Analysis and Performance Metrics

Observed Training Behavior

The training dynamics of the two architectures differed significantly, revealing an important property of from-scratch training on limited datasets.

VGG16 (45 epochs) exhibited stable and monotonic convergence. The training accuracy rose smoothly from approximately 60% in epoch 1 to approximately 99% by epoch 45. The validation accuracy correspondingly rose from approximately 91% to approximately 97%, with only minor oscillations. This behavior is consistent with VGG16's straightforward stacked convolutional design, which provides a well-conditioned gradient flow from random initialization.

MobileNetV2 (50 epochs) exhibited a markedly different convergence pattern. For the first 30 epochs, the validation accuracy remained fixed at approximately **50%** — equivalent to random binary guessing — while the training accuracy continued to rise steadily. This phenomenon indicates that the model was learning internal representations in the convolutional base that had not yet generalized sufficiently to the validation distribution. A sharp phase transition occurred around epoch 30–35, after which the validation accuracy rapidly increased to approximately **97.5%** and stabilized. This delayed generalization is a known characteristic of lightweight architectures with depthwise separable convolutions when trained without pre-trained weights: the gradient signals must first propagate through the bottleneck structures before meaningful validation-level features emerge.

Feature	**VGG16**	**MobileNetV2**
Model Size	Large	Lightweight
Training Speed	Slower	Faster per epoch
Epochs Trained	45	50
Convergence Pattern	Stable, Monotonic	Delayed Phase Transition
Memory Consumption	High	Low
Computational Cost	High	Low
Real-Time Performance	Moderate	Excellent
Deployment on Mobile Devices	Difficult	Easy

Table 1: Qualitative Comparison of VGG16 and MobileNetV2 Models.

Architecture	**Parameters**	**Epochs**	**Training Accuracy**	**Validation Accuracy**
VGG16	$\approx 14.7M$	45	$\approx 99\%$	$\approx 97\%$
MobileNetV2	$\approx 2.2M$	50	$\approx 99.3\%$	$\approx 97.5\%$

Table 2: Quantitative Performance Metrics (Final Epoch Values).

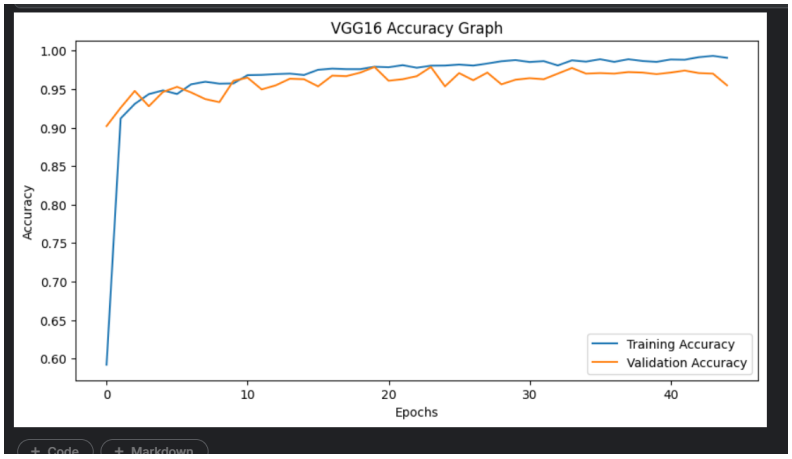

Figure 7: Training and Validation Accuracy curves for VGG16 (45 epochs). Both curves exhibit stable, monotonic convergence. Training accuracy rises from approximately 60% to 99%, while validation accuracy stabilizes between 96–97%, demonstrating consistent generalization throughout the training process.

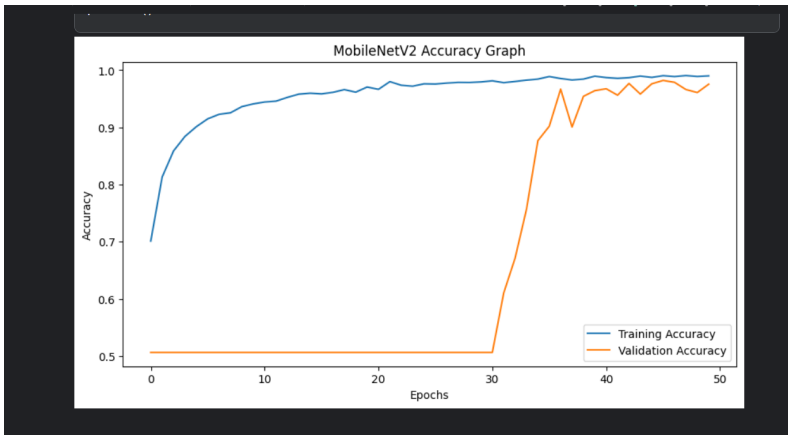

Figure 8: Training and Validation Accuracy curves for MobileNetV2 (50 epochs). The validation accuracy remains near 50% for the first 30 epochs, exhibiting a characteristic delayed phase transition before rapidly converging to approximately 97.5% in the final epochs. Training accuracy increases steadily throughout, indicating successful feature learning in the convolutional base prior to generalization.

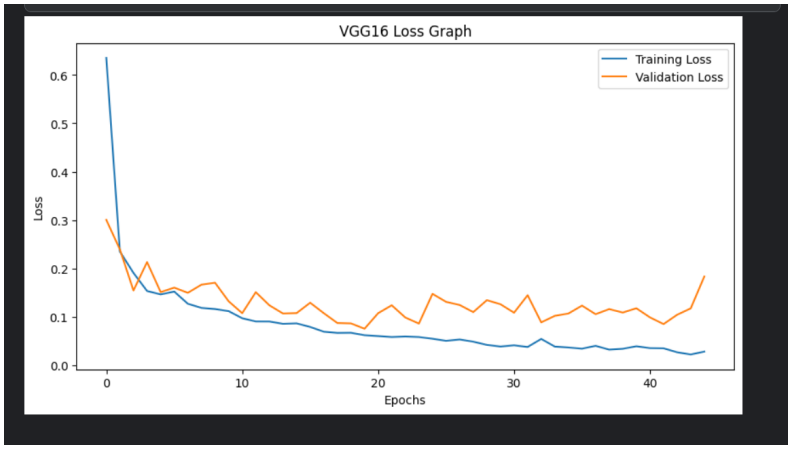

Figure 9: Training and Validation Loss curves for VGG16 (45 epochs). Training loss decreases smoothly from 0.64 to approximately 0.03. Validation loss converges to approximately 0.10–0.13, with minor oscillations, indicating stable optimization without significant overfitting.

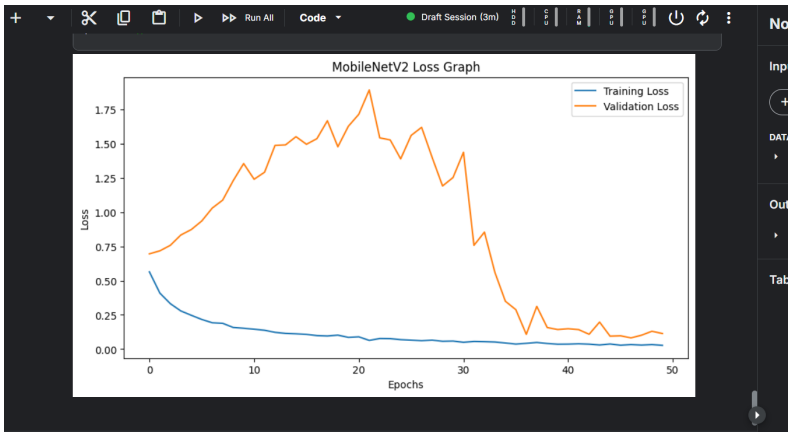

Figure 10: Training and Validation Loss curves for MobileNetV2 (50 epochs). Training loss decreases monotonically from 0.58 to approximately 0.03. Validation loss initially rises sharply, peaking at approximately 1.90 around epoch 22, before rapidly decreasing post epoch 30 — consistent with the delayed generalization observed in the accuracy curves.

4 Implementation and Deployment

4.1 System Implementation

The Face Mask Detection system was implemented using Python and several deep learning and computer vision libraries, including TensorFlow, Keras, OpenCV, NumPy, and Matplotlib. The implementation workflow consists of dataset preprocessing, from-scratch model construction, model training, evaluation, and real-time deployment.
The complete system pipeline is illustrated as follows:

1. Image Dataset Collection
2. Data Preprocessing and Augmentation
3. From-Scratch CNN Training using MobileNetV2 and VGG16
4. Model Training and Validation
5. Real-Time Face Detection using OpenCV
6. Mask Classification using Sigmoid Output

4.2 Real-Time Verification using OpenCV

For real-time testing and verification, the trained model was integrated with OpenCV. The webcam continuously captures live video frames, which are processed frame-by-frame. The implementation process includes:

1. Capturing video frames using OpenCV,
2. Resizing the image to 224×224,
3. Applying normalization and preprocessing,
4. Passing the processed image into the trained CNN model,
5. Predicting mask or no-mask using Sigmoid probability output.

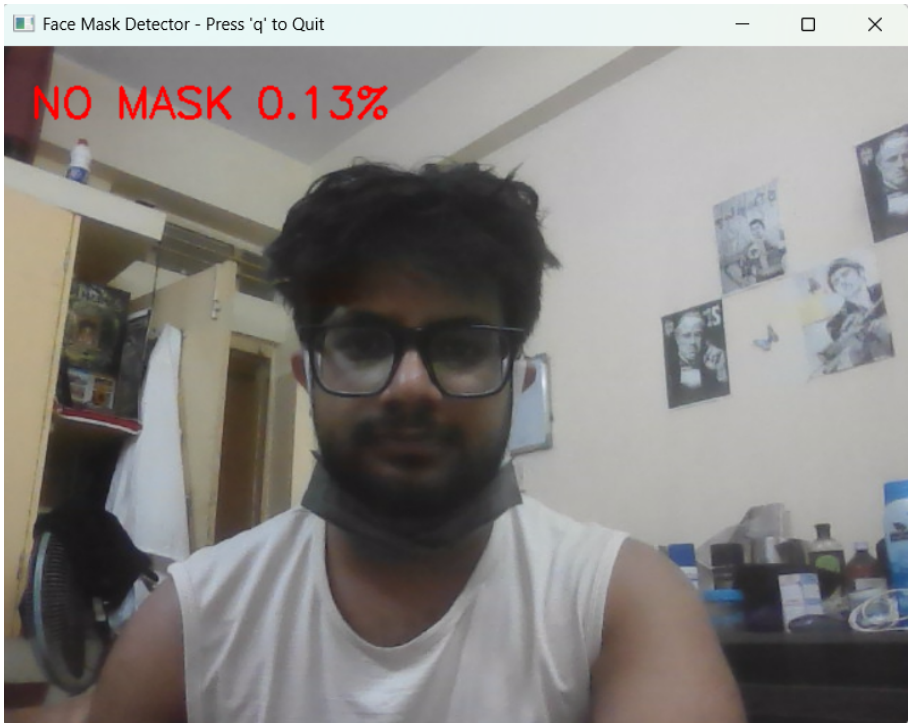

Figure 11: Real-time Testing: Without Mask Detection

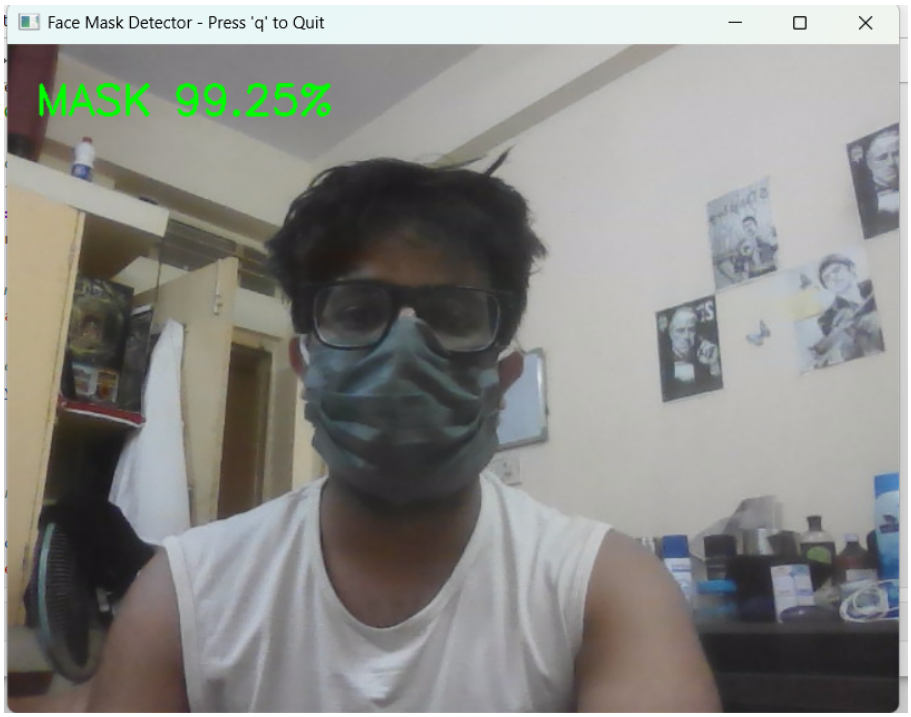

Figure 12: Real-time Testing: With Mask Detection

4.3 Deployment Strategies and Scalability

The trained Face Mask Detection model was deployed as an interactive web application using **Streamlit**, a high-level Python framework designed for rapid deployment of machine learning models. The deployment was carried out by hosting the application source code on **GitHub**, which was then connected to **Streamlit Cloud** for automated deployment.
The deployment pipeline consisted of the following components:

- **main.py** — The primary application script containing the Streamlit interface, image preprocessing logic, and model inference pipeline.
- **Saved Model File** — The trained `.h5` model file containing the learned weights of the CNN architecture.
- **requirements.txt** — A dependency file specifying all required Python libraries (TensorFlow, Keras, OpenCV, Streamlit, NumPy) and their versions, ensuring consistent runtime environment reproduction on the cloud server.
- **Python Version File** — A runtime configuration file specifying the Python version to ensure environment compatibility on Streamlit Cloud.

Upon connecting the GitHub repository to Streamlit Cloud, the platform automatically installs the specified dependencies and launches the web application, making it publicly accessible via a URL without any additional server configuration.
The live implementation is accessible at:

`https://face-mask-detection-b4vnmgsesqqr6r2wxdyjjz.streamlit.app/`

This interface supports real-time mask detection through image uploads, providing instant classification results using the trained VGG16 and MobileNetV2 models.

4.4 Result

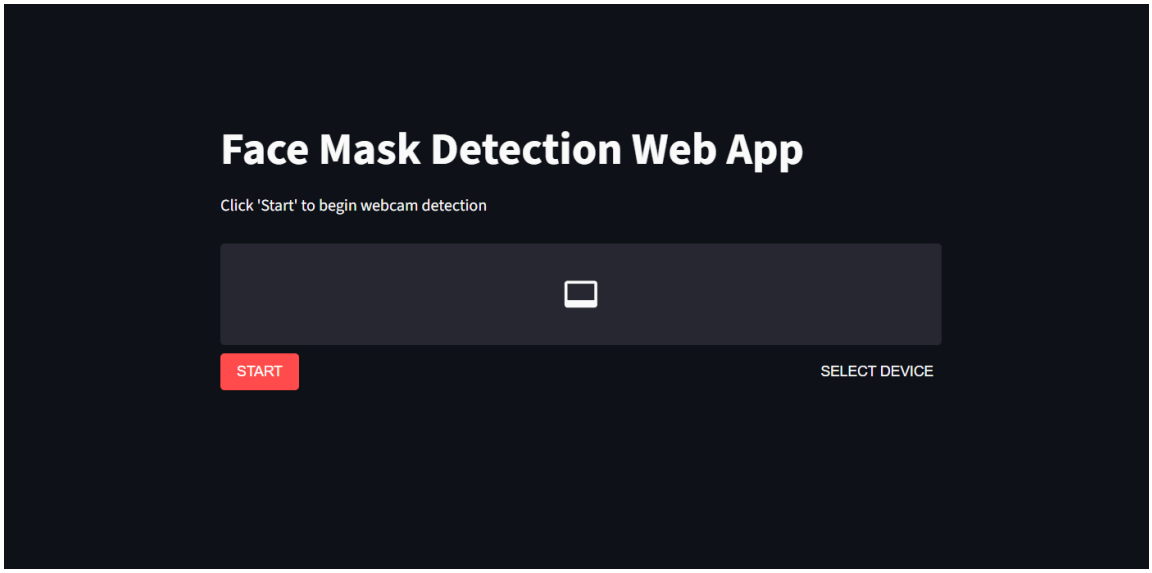

Figure 13: Web Application Interface: Initial Upload and Detection Result.

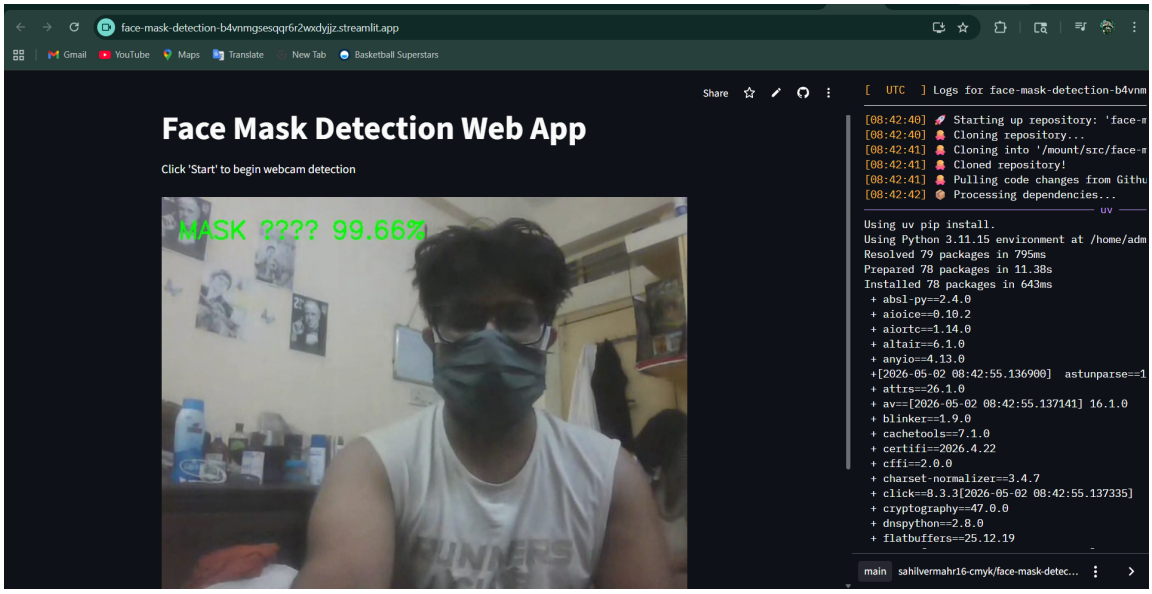

Figure 14: Web Application Interface: Real-time Inference Performance.

5 Conclusion

The development of this Face Mask Detection system successfully demonstrates the effectiveness — and the practical challenges — of training deep Convolutional Neural Networks **from scratch** for solving high-dimensional computer vision problems. By conducting a comparative analysis between **VGG16** (45 epochs) and **MobileNetV2** (50 epochs), both trained entirely on the face mask dataset without any pre-trained weights, this project provided critical insights into the convergence behavior and trade-offs of different architectural designs.

5.1 Summary of Findings

- **Convergence Behavior:** VGG16 demonstrated stable and monotonic convergence throughout training, achieving approximately 97% validation accuracy. MobileNetV2 exhibited a characteristic *delayed phase transition* — remaining near random-chance accuracy (50%) for the first 30 epochs before rapidly converging to approximately 97.5% validation accuracy. This behavior is attributed to the architectural properties of depthwise separable convolutions, which require more epochs to establish generalized gradient flow from random initialization.

- **Architectural Efficiency:** Despite its delayed generalization, MobileNetV2 ultimately achieved comparable accuracy to VGG16 with only ≈2.2M parameters versus ≈14.7M, confirming its efficiency advantage. Its faster per-epoch inference time makes it better suited for real-time deployment on resource-constrained devices.

- **Regularization and Generalization:** The implementation of Dropout (0.5) and stochastic data augmentation (rotation, horizontal flip) ensured that both models generalized effectively to unseen data, with final validation accuracies closely matching training accuracies.

- **Practical Utility:** The integration of the models with **OpenCV** and **Streamlit** transformed a theoretical deep learning framework into a functional web application, capable of providing real-time safety monitoring.

5.2 Limitations and Future Work

A key limitation observed in this study is the sensitivity of MobileNetV2 to training from random initialization on a relatively small dataset (≈8,000 images). The delayed generalization phenomenon, while ultimately resolved, significantly increases the required

training time and introduces risk of premature termination if training is stopped before the phase transition. Future iterations of this work could address these limitations through the following directions:

- **Transfer Learning Comparison:** A natural extension of this work would be a systematic comparison between from-scratch training (as conducted here) and fine-tuning from ImageNet pre-trained weights. This would directly quantify the accuracy and convergence benefits of pre-trained initialization, particularly for lightweight architectures such as MobileNetV2 on limited datasets.

- **Extended and Diverse Dataset:** The training dataset of ≈8,000 images proved sufficient for VGG16 but contributed to the delayed convergence observed in MobileNetV2. Incorporating a larger and more diverse dataset — covering varied lighting conditions, ethnicities, mask types, and occlusion levels — would improve robustness and reduce architecture-specific sensitivity.

- **Multi-Class Classification:** The current system performs binary classification (mask vs. no mask). A meaningful extension would be a three-class formulation: *With Mask*, *Without Mask*, and *Improper Mask* (e.g., mask worn below the nose). This would significantly increase the practical utility of the system in real-world enforcement scenarios.

- **Real-Time Webcam Integration and Mobile Deployment:** The current Streamlit deployment supports static image uploads. Integrating a live webcam stream directly into the web interface, combined with model quantization techniques (such as TensorFlow Lite conversion), would enable efficient real-time inference on mobile and edge devices — making the system deployable in resource-constrained environments such as entry checkpoints and public transport hubs.

References

[1] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510-4520.

[2] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv preprint arXiv:1409.1556.

[3] Chollet, F., et al. (2015). *Keras: Deep Learning for Humans*. GitHub repository: `https://github.com/keras-team/keras`.

[4] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.

[5] Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. arXiv preprint arXiv:1412.6980.

[6] Streamlit Inc. (2019). *Streamlit Documentation: Building and deploying data apps*. Available at: `https://docs.streamlit.io`.

[7] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. The Journal of Machine Learning Research, 15(1), 1929-1958.